Flujo máximo (y búsqueda local)

TDA / Algo3

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2do Cuatrimestre de 2026

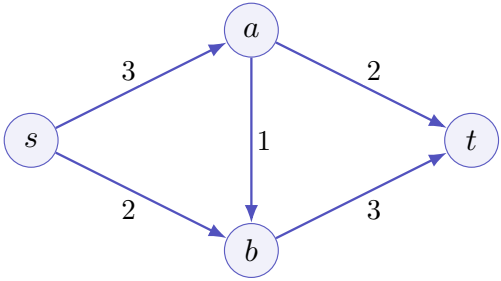
Plan

1. El problema de flujo máximo (y la técnica de búsqueda local).
2. Método de Ford y Fulkerson.
3. Cortes, correctitud y complejidad.
4. Resolviendo asignación con flujo.
5. El algoritmo de Edmonds y Karp.
6. Otro ejemplo de aplicación.
7. Conclusiones varias.

Flujo en una red

Se tiene una red ferroviaria que conecta un origen s (*source*) con un destino t (*target*) con ciertas restricciones:

- Cada tramo admite cierta cantidad de carga.
- La carga puede distribuirse entre distintas rutas.
- No se puede guardar carga en las estaciones intermedias.



Las etiquetas indican capacidades.

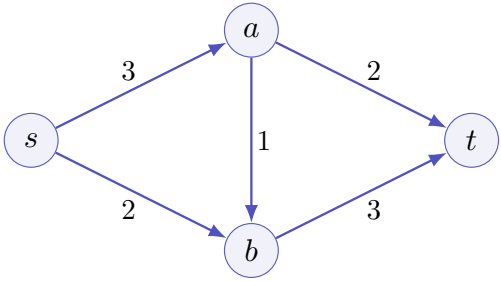
El problema

¿Cuánta carga podemos transportar de s a t?

Flujo en una red

Se tiene una red ferroviaria que conecta un origen s (*source*) con un destino t (*target*) con ciertas restricciones:

- Cada tramo admite cierta cantidad de carga.
- La carga puede distribuirse entre distintas rutas.
- No se puede guardar carga en las estaciones intermedias.



Las etiquetas indican capacidades.

El problema

¿Cuánta carga podemos transportar de s a t?

En este ejemplo, 5 unidades.

Flujo máximo

A este problema se lo conoce como **Flujo Máximo**. Más allá de ser interesante por si mismo, hay muchos otros problemas que pueden pensarse como un caso particular de flujo.

Asignación

Tenemos N personas y tareas, y cada persona puede hacer algún subconjunto de tareas.

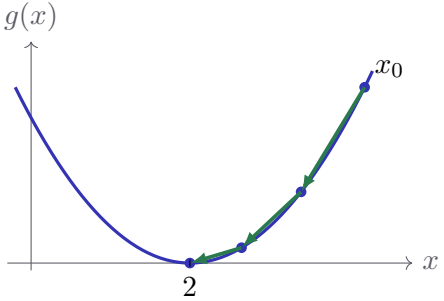
Luego, ¿Es posible asignar una tarea distinta a cada persona, con la condición de que cada persona sea asignada a una de las tareas que puede realizar?

Más adelante vamos a ver como este problema se **reduce** al de flujo máximo.

Flujo máximo

Aparte, el algoritmo para resolver flujo máximo que vamos a ver puede pensarse como la aplicación de la técnica de **búsqueda local**.

1. Empezar con una solución válida.
2. Buscar una modificación que la mejore.
3. Repetir hasta que no haya mejoras.



Ejemplo

Minimizar $g(x) = (x-2)^2$ (o, en general, cualquier función convexa).

Nuestro algoritmo de flujo va a empezar en una solución inicial “trivial” e iterativamente va a mejorarla.

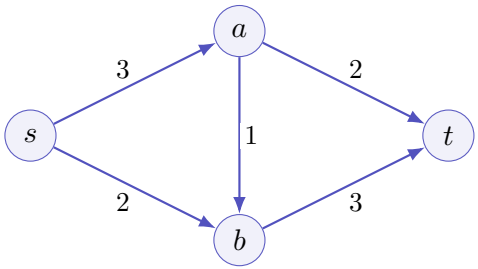
Flujo máximo

Formalicemos el problema. La entrada es una **red (de flujo)**

Flow network

Una **red de flujo** consiste en:

- Un grafo dirigido finito $G = (V, E)$.
- Dos vértices distintos: origen s y destino t.
- Una capacidad $c(e) \in \mathbb{R}_{\geq 0}$ para cada eje $e \in E$.



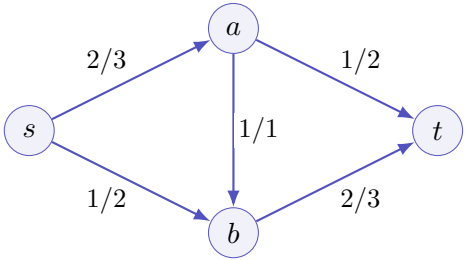
Un **flujo** es una función $f : E \to \mathbb{R}$ que asigna un número $f(e)$ a cada eje que respeta la **restricción de capacidad** y de **conservación**.

Flujo máximo

Capacidad

Para todo eje $(u, v) \in E$:

$$0 \leq f(u, v) \leq c(u, v).$$



Típicamente usamos etiquetas **flujo** / **capacidad** para indicar un flujo.

Por ejemplo, en $s \to a$ se usan 2 de las 3 unidades disponibles.

El eje $a \to b$ está **saturado**, ya que $f(a, b) = c(a, b) = 1$.

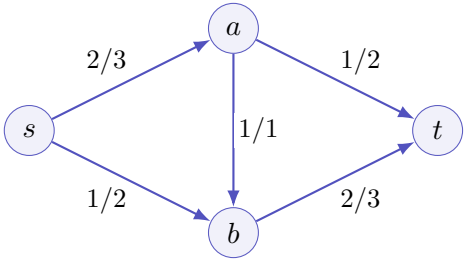
Restricción de conservación

Conservación en los vértices intermedios

Para todo $v \in V \setminus \{s, t\}$:

$$\sum_{(u,v)\in E} f(u, v) = \sum_{(v,w)\in E} f(v, w).$$

O sea, todo lo que entra, sale.



En a: entran 2 y salen $1 + 1$.

En b: entran $1 + 1$ y salen 2.

Las restricciones no exigen conservación en s ni en t.

Valor de un flujo

El flujo neto que sale de un vértice v dado un flujo f es

$$b_f(v) = \sum_{(v,w)\in E} f(v,w) - \sum_{(u,v)\in E} f(u,v).$$

El valor de un flujo f se define como lo que sale de s:

$$\mathrm{val}(f) = b_f(s)$$

El objetivo es encontrar un flujo de **valor máximo.**

Propiedad para ir calentando

Para todo flujo f vale que $b_f(s) = -b_f(t)$

Valor de un flujo

El flujo neto que sale de un vértice v dado un flujo f es

$$b_f(v) = \sum_{(v,w)\in E} f(v,w) - \sum_{(u,v)\in E} f(u,v).$$

El valor de un flujo f se define como lo que sale de s:

$$\mathrm{val}(f) = b_f(s)$$

El objetivo es encontrar un flujo de **valor máximo.**

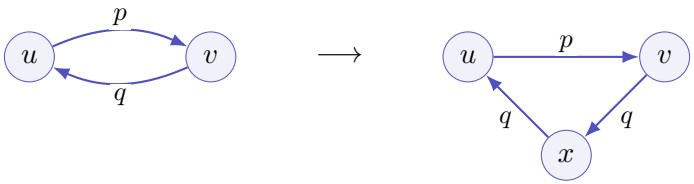
Propiedad para ir calentando

Para todo flujo f vale que $b_f(s) = -b_f(t)$

Resumen de la demo: por conservación es inmediato que $\sum_{v\in V\setminus\{s,t\}} b_f(v) = 0$. Pero el lado izquierdo es exactamente igual a $-b_f(s) - b_f(t)$ ya que los demás ejes se cancelan.

Algunas simplificaciones

Para simplificar, podemos asumir que nunca aparecen a la vez $u \to v$ y $v \to u$ en la red original.

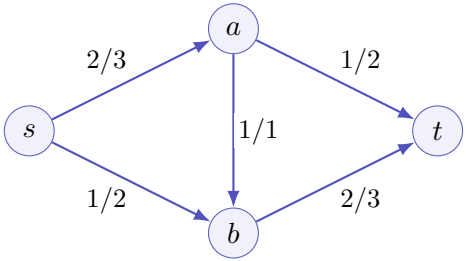


También podemos asumir que no hay ejes hacia s ni que salen de t.

Ford y Fulkerson

Metodo de Ford y Fulkerson

¿Cómo podemos mejorar un flujo?



Dado un eje $u \to v$ podríamos:

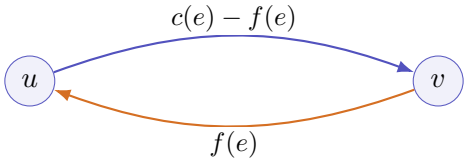
- Agregar hasta $c(u, v) - f(u, v)$ unidades.
- Deshacer hasta $f(u, v)$ unidades.

Sin embargo, al hacer esto podríamos perder la propiedad de conservación...

Para lidiar con esto, tenemos que introducir una estructura más, que es la clave de los algoritmos que vamos a ver hoy.

Red residual

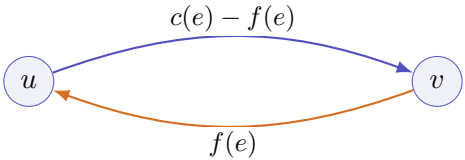
Dada una red G y un flujo f, vamos a armar la **red residual** G_f. Esta red tiene los mismos nodos que G, pero para elegir los ejes hacemos lo siguiente: por cada eje uv, colocamos dos ejes con las siguientes capacidades



- El eje de u a v en G_f representa el flujo que **todavía podemos mandar** de u a v.
- El eje de v a u representa el flujo que podemos **dejar de enviar** de u a v (que, en el fondo, es equivalente a mandar de vuelta el flujo de v a u).

No ponemos los ejes con capacidad nula.

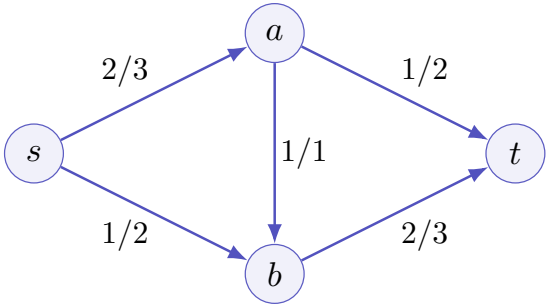
Red residual



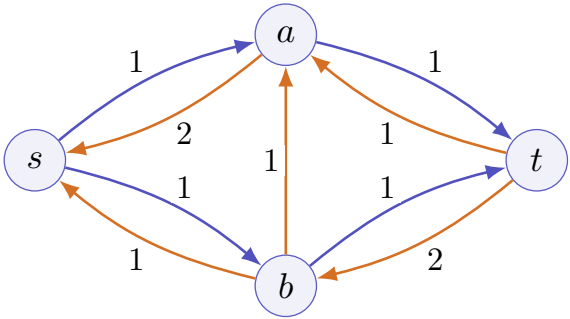
Escribimos $G_f = (V, E_f)$ y c_f para denotar las capacidades de esta red. Notar que $|E_f| \leq 2|E|$, por lo que la red es “chica”. Aparte, se puede armar en tiempo lineal.

La red residual

Red original: flujo / capacidad



Red residual: capacidad restante



Como el eje $a \to b$ está saturado solo aparece su inverso $b \to a$.

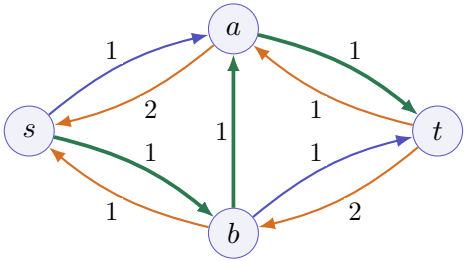
Camino de aumento

Para mejorar un flujo vamos a agregarle un **camino de aumento**.

Definición

Un **camino de aumento** es un camino simple de s a t en G_f. Su cuello de botella (o peso) es

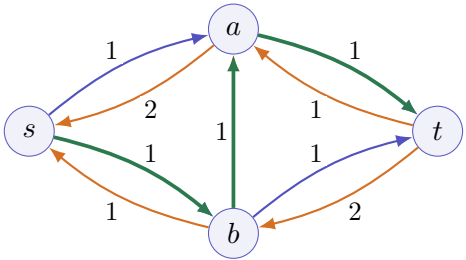
$$\Delta = \min_{(u,v)\in P} c_f(u,v) > 0.$$



Por ejemplo, $P = s \to b \to a \to t$.

Sus capacidades residuales son $1, 1, 1$, por lo tanto $\Delta = 1$.

Camino de aumento



Por ejemplo, $P = s \to b \to a \to t$.

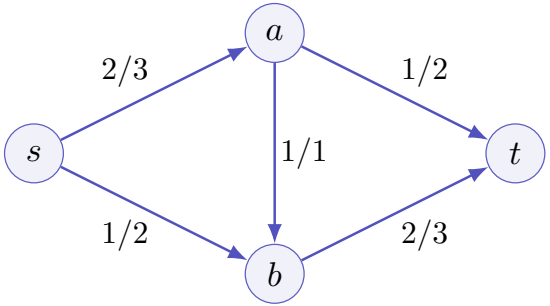
Sus capacidades residuales son $1, 1, 1$, por lo tanto $\Delta = 1$.

Podemos mejorar el flujo f usando el camino P: si uv es un eje de P y este era un eje de la red, entonces le agregamos Δ de flujo. Por otro lado, si vu era el eje de la red, entonces le sacamos Δ. Más formalmente, definimos un flujo f' como

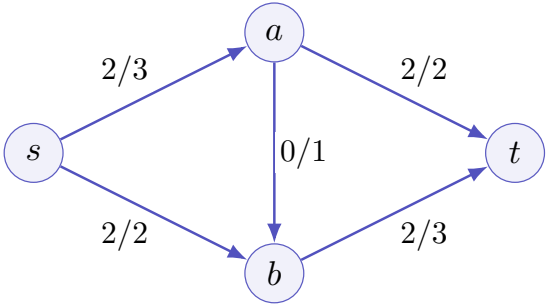
$$f'(u,v) = \begin{cases} f(u,v) + \Delta & uv \in P \\ f(u,v) - \Delta & vu \in P \\ f(u,v) & \text{caso contrario} \end{cases}$$

Camino de aumento

Antes: valor 3



Después: valor 4



$$f'(s,b) = 1 + 1 = 2, \quad f'(a,b) = 1 - 1 = 0, \quad f'(a,t) = 1 + 1 = 2.$$

¿Vale que f' es un flujo válido?

Camino de aumento

Veamos que f' cumple las dos restricciones.

- **Capacidad.** Si se agrego Δ al flujo de u a v, por definición $\Delta \leq c(uv) - f(uv)$. Si se sacó de uv, por definición $\Delta \leq f(uv)$. Luego $0 \leq f'(uv) \leq c(uv)$.
- **Conservación.** Si el camino para por u entonces se suma Δ a lo que entra pero luego se resta Δ. Por lo tanto, lo que entra sigue siendo igual a lo que sale.

Aparte, se tiene que

$$\mathrm{val}(f') = \mathrm{val}(f) + \Delta$$

¿Cómo y en qué complejidad puede encontrarse un camino de aumento?

Camino de aumento

Veamos que f' cumple las dos restricciones.

- **Capacidad.** Si se agrego Δ al flujo de u a v, por definición $\Delta \leq c(uv) - f(uv)$. Si se sacó de uv, por definición $\Delta \leq f(uv)$. Luego $0 \leq f'(uv) \leq c(uv)$.
- **Conservación.** Si el camino para por u entonces se suma Δ a lo que entra pero luego se resta Δ. Por lo tanto, lo que entra sigue siendo igual a lo que sale.

Aparte, se tiene que

$$\mathrm{val}(f') = \mathrm{val}(f) + \Delta$$

¿Cómo y en qué complejidad puede encontrarse un camino de aumento? $O(n + m)$ usando DFS o BFS. Aparte, aumentar luego el flujo de f puede hacerse en $O(n)$ (es solo recorrer las aristas del camino).

Método de Ford–Fulkerson

Con esto ya podemos dar el algoritmo de FF:

FORD–FULKERSON(G, c, s, t)

$f(e) \leftarrow 0$ para todo $e \in E$
repetir:
  $G_f \leftarrow$ RESIDUAL(G, c, f)
  $P \leftarrow$ CAMINO(G_f, s, t)
  si $P = \perp$: **devolver** f
  $f \leftarrow$ AUMENTAR(f, P)

- El algoritmo inicia con un flujo trivial, y en cada paso busca un camino de aumento para mejorarlo.
- Cada iteración toma $O(n + m)$.
- Pero... ¿Cuántas iteraciones hace? ¿Cómo sabemos que encuentra un flujo máximo?

Para probar correctitud vamos a tener que hablar de otro problema: **Corte mínimo**.

Corte mínimo y correctitud de FF

Corte mínimo y correctitud de FF

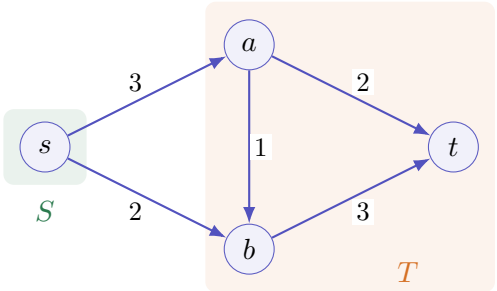
Cortes de una red

Demos primero la definición formal de corte $s - t$.

Corte s–t

Un corte es una partición (S, T) de V con $s \in S$ y $t \in T$. El valor o capacidad del corte es

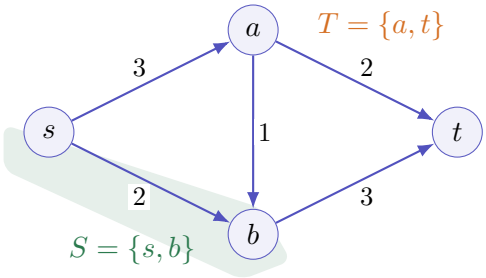
$$c(S,T) = \sum_{\substack{(u,v)\in E \\ u\in S,\ v\in T}} c(u,v).$$



Aquí $S = \{s\}$ y $T = \{a, b, t\}$, con capacidad $3 + 2 = 5$.

Un **corte mínimo** minimiza $c(S, T)$ entre todos los cortes s–t.

Cortes de una red



Los ejes de S hacia T son $s \to a$ y $b \to t$.

$$c(S, T) = 3 + 3 = 6.$$

El eje $a \to b$ cruza en sentido contrario y **no suma** a la capacidad del corte.

Relación entre flujos y cortes

Dados dos conjuntos $A, B \subseteq V$ y un flujo f, usamos

$$f(A, B) = \sum_{\substack{(u,v) \in E \\ u \in A,\ v \in B}} f(u, v).$$

para denotar el flujo total que va de A a B. Vale la siguiente propiedad.

Lema

Para todo corte (S, T) y todo flujo válido f:

$$\text{val}(f) = f(S, T) - f(T, S).$$

Relación entre flujos y cortes

Dados dos conjuntos $A, B \subseteq V$ y un flujo f, usamos

$$f(A, B) = \sum_{\substack{(u,v)\in E \\ u\in A,\ v\in B}} f(u, v).$$

para denotar el flujo total que va de A a B. Vale la siguiente propiedad.

Lema

Para todo corte (S, T) y todo flujo válido f:

$$\mathrm{val}(f) = f(S, T) - f(T, S).$$

Resumen de la demo: Sumamos $b_f(v)$ sobre $v \in S$. Este valor es igual a $b_f(s) = \mathrm{val}(f)$ por conservación. Aparte, si cancelamos las aristas solo nos quedan las que salen de S con valor positivo y las que entran a S con valor negativo (y esas son las que van de T a S).

Relación entre flujos y cortes

Por el lema, la restricción de capacidad y la no negatividad del flujo tenemos que

$$\begin{aligned} \text{val}(f) &= f(S,T) - f(T,S) \\ &\leq f(S,T) \\ &\leq c(S,T). \end{aligned}$$

Relación entre flujo máximo y corte mínimo

$$\max_f \text{val}(f) \ \leq \ \min_{(S,T)} c(S,T).$$

Por lo tanto, para probar que un flujo es máximo alcanza con ver que tiene el mismo valor que un corte.

Teorema de flujo máximo y corte mínimo

Teorema

Para todo flujo válido f, son equivalentes:

1. f es un flujo máximo.
2. No hay caminos de aumento en G_f.
3. $\text{val}(f) = \min_{(S,T)} c(S,T)$.

Teorema de flujo máximo y corte mínimo

Teorema

Para todo flujo válido f, son equivalentes:

1. f es un flujo máximo.
2. No hay caminos de aumento en G_f.
3. $\text{val}(f) = \min_{(S,T)} c(S,T)$.

$(1) \Rightarrow (2)$**.** Si hubiera un camino de aumento, podríamos obtener un flujo válido de valor estrictamente mayor.

Teorema de flujo máximo y corte mínimo

Teorema

Para todo flujo válido f, son equivalentes:

1. f es un flujo máximo.
2. No hay caminos de aumento en G_f.
3. $\text{val}(f) = \min_{(S,T)} c(S,T)$.

$(1) \Rightarrow (2)$**.** Si hubiera un camino de aumento, podríamos obtener un flujo válido de valor estrictamente mayor.

$(3) \Rightarrow (1)$**.** Ningún flujo puede superar la capacidad de un corte mínimo.

Teorema de flujo máximo y corte mínimo

Teorema

Para todo flujo válido f, son equivalentes:

1. f es un flujo máximo.
2. No hay caminos de aumento en G_f.
3. $\text{val}(f) = \min_{(S,T)} c(S,T)$.

$(1) \Rightarrow (2)$**.** Si hubiera un camino de aumento, podríamos obtener un flujo válido de valor estrictamente mayor.

$(3) \Rightarrow (1)$**.** Ningún flujo puede superar la capacidad de un corte mínimo.

Ahora hacemos la otra, que es la difícil.

Teorema de flujo máximo y corte mínimo

Sea S el conjunto de vértices alcanzables desde s en G_f, y $T = V \setminus S$. Como no hay camino de aumento, $t \notin S$.

- Si $u \in S$, $v \in T$ y $(u, v) \in E$, entonces $f(u, v) = c(u, v)$. De lo contrario, el eje residual directo permitiría alcanzar v.
- Si $u \in T$, $v \in S$ y $(u, v) \in E$, entonces $f(u, v) = 0$. De lo contrario, el eje residual inverso permitiría alcanzar u.

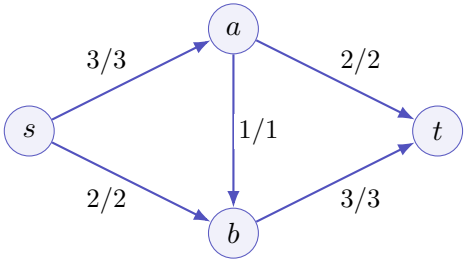
Por lo tanto,

$$\text{val}(f) = f(S, T) - f(T, S) = c(S, T) - 0 = c(S, T).$$

Por lo tanto f es un flujo máximo (ya que coincide con un corte). Aparte, ese corte tiene que ser mínimo (por el mismo motivo).[1]

[1] ¡Notar que en esta demo dimos un algoritmo para encontrar un corte mínimo!

Teorema de flujo máximo y corte mínimo



Después de mandar flujo por el camino de aumento, la red residual queda sin ejes. En particular, el corte $\{s\}$ contra $\{a, b, t\}$ es mínimo.

Con todo esto podemos concluir que FF es correcto **si termina**. Veamos entonces condiciones para su terminación.

El caso de capacidades naturales

Supongamos $c(e) \in \mathbb{N}$ para todo eje, y sea F el valor del flujo máximo. Vale que:

1. El flujo inicial es entero.
2. Si el flujo es entero, todas las capacidades residuales son enteras.
3. Cada cuello de botella $\Delta > 0$ es entero, luego $\Delta \geq 1$.
4. El aumento conserva la integridad y sube el valor al menos en 1.

¿Qué podemos decir entonces con respecto a la cantidad de iteraciones del algoritmo?

El caso de capacidades naturales

Supongamos $c(e) \in \mathbb{N}$ para todo eje, y sea F el valor del flujo máximo. Vale que:

1. El flujo inicial es entero.
2. Si el flujo es entero, todas las capacidades residuales son enteras.
3. Cada cuello de botella $\Delta > 0$ es entero, luego $\Delta \geq 1$.
4. El aumento conserva la integridad y sube el valor al menos en 1.

¿Qué podemos decir entonces con respecto a la cantidad de iteraciones del algoritmo?

Cota para el caso entero

Hay a lo sumo F aumentos.

El caso de capacidades naturales

Esto nos permite deducir una cota de complejidad para el caso en que las capacidades sean enteras:

El caso de capacidades naturales

Esto nos permite deducir una cota de complejidad para el caso en que las capacidades sean enteras:

Cota total con capacidades enteras

Hay a lo sumo F aumentos. Luego, la complejidad queda:

$$O\big(mF\big)$$

- La complejidad espacial es $O(n+m) = O(m)$.

Asignación

Ejemplo de aplicación: el problema de asignación

El problema de asignación

Hay N personas y N tareas. Conocemos qué personas pueden realizar cada tarea.

Objetivo

Asignar a cada persona una tarea que pueda hacer, sin asignar la misma tarea a dos personas.

Persona	Tareas posibles
1	A, B
2	A
3	B, C

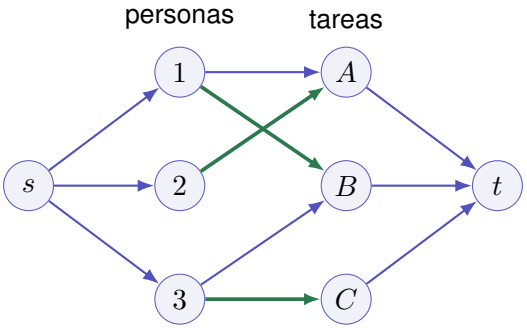
Una asignación válida es

$1 \mapsto B, \quad 2 \mapsto A, \quad 3 \mapsto C.$

Vamos a dar un algoritmo que convierte una instancia del problema de **asignación** en una instancia del problema de **flujo máximo**. A este tipo de transformaciones se las llama **reducciones**

Asignación

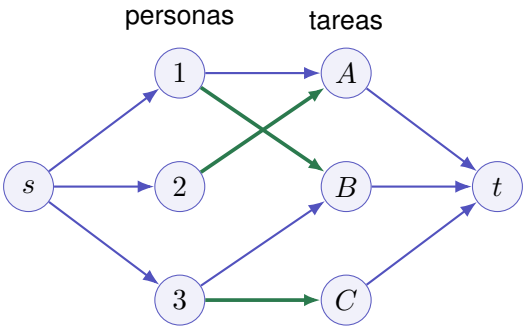
Vamos a armar una red (bipartita) de la siguiente forma:



- $s \to p_i$ para cada persona.
- $p_i \to q_j$ si i puede realizar j.
- $q_j \to t$ para cada tarea.

Todas las capacidades son 1.

Asignación



- $s \to p_i$ para cada persona.
- $p_i \to q_j$ si i puede realizar j.
- $q_j \to t$ para cada tarea.

Todas las capacidades son 1.

La intuición es que cada **unidad de flujo** se va a corresponder con una asignación. Aparte, a cada persona solo le entra una unidad (o sea, se le va a asignar una única tarea), y cada tarea solo puede recibir una persona (o sea, cada tarea es realizada por una única persona). Formalicemos estas afirmaciones y demostrémoslas.

Asignación

Para cada persona p_i con $1 \leq i \leq N$ sea $T_i \subseteq \{t_1 \dots t_N\}$ es conjunto de tareas que puede realizar. Decimos que una asignación a de personas a tareas es válida si

- $a(p_i) \in T_i$ para todo i.
- Para cada t_j, vale que $|a^{-1}(t_j)| = 1$.

Sea R_T la red que se arma a partir de los $T_1 \dots T_N$. Queremos probar la siguiente afirmación

Correctitud de la reducción

Existe una asignación compatible con $T_1 \dots T_N$ si y solamente si hay un flujo en R_T de valor al menos N.

Asignación

Probemos primero la ida: si hay una asignación válida a... ¿Cómo probamos que el valor del flujo máximo es al menos N?

Asignación

Probemos primero la ida: si hay una asignación válida a... ¿Cómo probamos que el valor del flujo máximo es al menos N?
Armamos el flujo usando a. Lo definimos como

$$f(p_i, t_j) = \begin{cases} 1 & a(p_i) = t_j \\ 0 & \text{caso contrario} \end{cases}$$

Aparte, el flujo manda 1 por cada arista saliente de s y entrante a t. ¿Este flujo es válido y tiene valor al menos N?

Asignación

Probemos primero la ida: si hay una asignación válida a... ¿Cómo probamos que el valor del flujo máximo es al menos N?
Armamos el flujo usando a. Lo definimos como

$$f(p_i, t_j) = \begin{cases} 1 & a(p_i) = t_j \\ 0 & \text{caso contrario} \end{cases}$$

Aparte, el flujo manda 1 por cada arista saliente de s y entrante a t. ¿Este flujo es válido y tiene valor al menos N?
Satisface las restricciones de capacidad y tiene valor exactamente N. Para ver conservación, notemos que a cada nodo p_i de la primera columna le llega una unidad de flujo, y también le sale solo una dirigida hacia $a(p_i)$. Luego, para t_j de la segunda columna le sale una unidad de flujo hacia t, y le entra un flujo igual a $|a^{-1}(t_j)| = 1$.

Asignación

Para la vuelta, tenemos que probar que si hay un flujo de valor al menos N, entonces hay una asignación válida ¿Cómo probamos esto?

Asignación

Para la vuelta, tenemos que probar que si hay un flujo de valor al menos N, entonces hay una asignación válida ¿Cómo probamos esto?
Sea f un flujo de valor N (que es máximo, ¿Por qué?) y entero (¿Por qué puedo asumir esto?). Defino una asignación a como $a(p_i) = t_j$ tal que $f(p_i, t_j) = 1$. Notemos que tiene que existir alguna arista saliente de p_i con flujo: caso contrario, f no podría tener valor N.

Asignación

Para la vuelta, tenemos que probar que si hay un flujo de valor al menos N, entonces hay una asignación válida ¿Cómo probamos esto?
Sea f un flujo de valor N (que es máximo, ¿Por qué?) y entero (¿Por qué puedo asumir esto?). Defino una asignación a como $a(p_i) = t_j$ tal que $f(p_i, t_j) = 1$. Notemos que tiene que existir alguna arista saliente de p_i con flujo: caso contrario, f no podría tener valor N. Claramente esta asignación cumple la propiedad de que $a(p_i) \in T_i$ por construcción.
Falta ver que $|a^{-1}(t_j)| = 1$. Para esto, por conservación, sabemos que el flujo que entra a cada t_j tiene que ser igual al que sale, que es 1. Por lo tanto, hay solo una persona que puede estar mandando flujo hacia t_j. Por lo tanto, concluímos que la asignación es válida.

Complejidad

Si $q = \sum_i |T_i|$, vale que

$$n = 2N + 2, \qquad m = 2N + q = O(N^2), \qquad F \leq N.$$

Notemos que la red se puede armar en tiempo lineal. Luego, la complejidad está dominada por la ejecución de FF, que tiene complejidad

$$O\big(mF\big) = O\big(N^3\big)$$

Edmonds-Karp

Edmonds-Karp

Edmonds-Karp

Si usamos BFS para obtener el camino de aumento, entonces tenemos el algoritmo de EK.

Edmonds-Karp

En cada iteración, elegir un camino de aumento **de longitud mínima**.

Algoritmo

$f \leftarrow 0$.

Mientras BFS encuentre un camino P de s a t en G_f:

$\Delta \leftarrow \min\{c_f(e) : e \in P\}$.

Aumentar f en Δ a lo largo de P.

Devolver f.

- ¿Este algoritmo es correcto?

Edmonds-Karp

Si usamos BFS para obtener el camino de aumento, entonces tenemos el algoritmo de EK.

Edmonds-Karp

En cada iteración, elegir un camino de aumento **de longitud mínima**.

Algoritmo

$f \leftarrow 0$.

Mientras BFS encuentre un camino P de s a t en G_f:

$\Delta \leftarrow \min\{c_f(e) : e \in P\}$.

Aumentar f en Δ a lo largo de P.

Devolver f.

- ¿Este algoritmo es correcto? Si, la prueba de FF era agnostica al camino que se elegía en cada paso.
- Lo interesante es que podemos acotar la cantidad de iteraciones por una expresión que solo depende de n y m.

Edmonds-Karp

Para deducir esta cota primero vamos a tener que introducir algunos lemas. Sean $d(v) = \text{dist}_{G_f}(s, v)$ y $d'(v) = \text{dist}_{G_{f'}}(s, v)$, antes y después de actualizar la red residual por un camino BFS P. Vale lo siguiente:

Lema

Para todo vértice v, se cumple $d'(v) \geq d(v)$.

Edmonds-Karp

Para deducir esta cota primero vamos a tener que introducir algunos lemas. Sean $d(v) = \text{dist}_{G_f}(s, v)$ y $d'(v) = \text{dist}_{G_{f'}}(s, v)$, antes y después de actualizar la red residual por un camino BFS P. Vale lo siguiente:

Lema

Para todo vértice v, se cumple $d'(v) \geq d(v)$.

Estudiemos un eje (u, v) de la **nueva** red residual:

1. Si ya estaba en G_f, la definición de distancia da
$$d(v) \leq d(u) + 1.$$
2. Si acaba de aparecer, es el inverso de un eje (v, u) de P. Como P es un camino más corto, sus prefijos también lo son:
$$d(u) = d(v) + 1 \quad \Longrightarrow \quad d(v) = d(u) - 1 \leq d(u) + 1.$$

Edmonds-Karp

Tomemos ahora cualquier camino en $G_{f'}$

$$s = x_0 \longrightarrow x_1 \longrightarrow \cdots \longrightarrow x_\ell = v.$$

Como $d(x_0) = 0$, por inducción sobre j:

$$d(x_j) \leq d(x_{j-1}) + 1 \leq j.$$

En particular, $d(v) \leq \ell$. Aplicándolo a un camino nuevo más corto:

$$\boxed{d(v) \leq d'(v).}$$

Obs: Si v era inalcanzable no puede volverse alcanzable.

Edmonds-Karp

Cada vez que encontramos un camino de aumento, hay por lo menos un eje que desaparece tras actualizar la red. A esos ejes los llamamos **criticos**.

Eje crítico

Un eje residual (u, v) del camino elegido es **crítico** si su capacidad es Δ.

Edmonds-Karp

Cada vez que encontramos un camino de aumento, hay por lo menos un eje que desaparece tras actualizar la red. A esos ejes los llamamos **criticos**.

Eje crítico

Un eje residual (u, v) del camino elegido es **crítico** si su capacidad es Δ.

Estos ejes son relevante por la siguiente propiedad: supongamos que (u, v) es crítico en las iteraciones i y j, con $i < j$.

1. Por como se eligen los caminos, en la iteración i ocurre que $d_i(v) = d_i(u) + 1$.
2. Para volver a aparecer en la red residual en alguna iteración k, con $i < k < j$, el camino BFS debe usar el eje inverso (v, u). Luego, vale que $d_k(u) = d_k(v) + 1$.
3. Por la monotonía de las distancias tenemos entonces:

$$\begin{aligned} d_j(u) &\geq d_k(u) = d_k(v) + 1 \\ &\geq d_i(v) + 1 = d_i(u) + 2. \end{aligned}$$

Conclusión: entre dos veces que un mismo eje es crítico su distancia a s aumenta.

Edmonds-Karp

1. **Corolario**: un eje puede ser crítico a lo sumo n veces.
2. En cada iteración hay al menos un eje crítico.

¿Qué cota obtenemos entonces para la cantidad de iteraciones?

Edmonds-Karp

1. **Corolario**: un eje puede ser crítico a lo sumo n veces.
2. En cada iteración hay al menos un eje crítico.

¿Qué cota obtenemos entonces para la cantidad de iteraciones?

Cota de iteraciones de EK

Hay $O(nm)$ iteraciones.

Como cada iteración cuesta $O(m)$, la complejidad final queda $O(nm^2)$.

Edmonds-Karp

Si las capacidades son enteras la cota que vimos para FF también aplica a EK. Luego, la complejidad queda

$$O(\min\{Fm, nm^2\})$$

Aplicación: Selección de proyectos con dependencias

Se tienen N proyectos, y cada proyecto i tiene una ganancia neta entera w_i, (que puede ser negativa). Aparte, hay un conjunto de dependencias de la forma (i, j) que indica que si elegimos i también debemos hacer j.

Problema

Elegir un conjunto X de proyectos que maximice $\sum_{i \in X} w_i$, respetando las dependencias: si elegimos i y i requiere j, también debemos elegir j.

Proyecto	Ganancia
A	$+8$
B	-5
C	$+4$
D	-2

- A requiere B.
- C requiere B y D.

Los costos negativos sirven para modelar compras de insumos o infraestructura.

Aplicación: Selección de proyectos con dependencias

Proyectos (que suman) elegidos	Mejor conjunto factible	Ganancia
Ninguno	$\varnothing$	0
Solo A	$\{A, B\}$	$8 - 5 = 3$
Solo C	$\{B, C, D\}$	$4 - 5 - 2 = -3$
A y C	$\{A, B, C, D\}$	$8 + 4 - 5 - 2 = 5$

Aplicación: Selección de proyectos con dependencias

Proyectos (que suman) elegidos	Mejor conjunto factible	Ganancia
Ninguno	$\varnothing$	0
Solo A	$\{A, B\}$	$8 - 5 = 3$
Solo C	$\{B, C, D\}$	$4 - 5 - 2 = -3$
A y C	$\{A, B, C, D\}$	$8 + 4 - 5 - 2 = 5$

Vamos a modelarlo con **corte mínimo**, que en el fondo es **flujo máximo**.

Selección de proyectos con dependencias

Vamos a explotar el siguiente truco: si tenemos un nodo que representa a la tarea A y otro que representa a B, tal que A depende de B, si ponemos un eje con capacidad ∞ de A a B entonces A y B **tienen que estar juntos en el corte mínimo**.

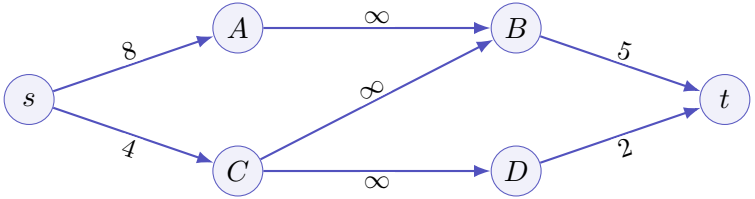
Selección de proyectos con dependencias

Vamos a explotar el siguiente truco: si tenemos un nodo que representa a la tarea A y otro que representa a B, tal que A depende de B, si ponemos un eje con capacidad ∞ de A a B entonces A y B **tienen que estar juntos en el corte mínimo**.
Luego, para encontrar la elección de tareas óptima vamos a ver cuáles proyectos quedaron **en el corte que contiene a** s.

Selección de proyectos

Armemos una posible red, siguiendo la idea que mencionamos antes:

- Si $w_i > 0$, agregamos $s \to i$ con capacidad w_i. Intuición: como el corte **minimiza**, los i con w_i muy grande van a "querer" pertenecer a la componente de s.
- Si $w_i < 0$, agregamos $i \to t$ con capacidad $-w_i$. Misma intuición: si un costo w_i es muy alto, vamos a querer dejarlo con t, es decir, no pagarlo.
- Si i requiere j, agregamos $i \to j$ con capacidad ∞.



Cómo interpretar un corte (S, T)

Elegimos exactamente los proyectos del lado de s: $X = S \setminus \{s\}$.

Selección de tareas con dependencias

Propiedad: Todo corte mínimo induce un conjunto de tareas factible.

Selección de tareas con dependencias

Propiedad: Todo corte mínimo induce un conjunto de tareas factible.*Demostración*: el corte $S = \{s\}$ tiene costo $< \infty$, por lo que el corte mínimo tiene costo menor a infinito. Luego, si ese corte incluye una tarea i, también tiene que incluir a sus dependencias.

Selección de tareas con dependencias

Propiedad: Todo corte mínimo induce un conjunto de tareas factible.*Demostración*: el corte $S = \{s\}$ tiene costo $< \infty$, por lo que el corte mínimo tiene costo menor a infinito. Luego, si ese corte incluye una tarea i, también tiene que incluir a sus dependencias.
Incluso podemos dar una expresión para el valor de un corte válido con peso menor a infinito:

Valor de un corte válido

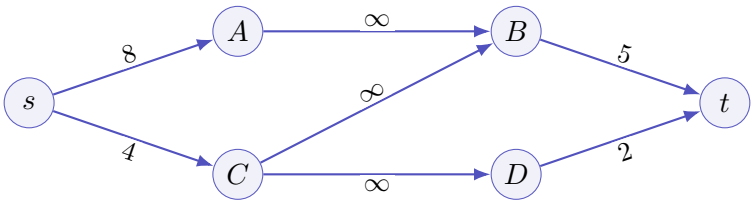
Si S es un corte con peso menor a infinito, las aristas que definen su peso son las que van de s a tareas rentables no elegidas y lo mismo desde los costos elegidos. Luego,

$$\begin{aligned} c(S,T) &= \sum_{\substack{w_i>0 \\ i\notin X}} w_i + \sum_{\substack{w_i<0 \\ i\in X}} (-w_i) \\ &= P - \sum_{i\in X} w_i. \end{aligned}$$

donde P es la suma de todos los w_i

Es decir, el corte mínimo **minimiza** esa expresión, por lo que **maximiza** el valor del conjunto X.

En el ejemplo



El corte mínimo es $S = \{A, C, B, D\}$, y eso se corresponde con hacer todas las tareas. El beneficio que se obtiene es 5.

Complejidad

Si N es la cantidad de proyectos y R la cantidad de dependencias: ¿Qué tamaño tiene la red? ¿Se puede acotar su flujo?

Complejidad

Si N es la cantidad de proyectos y R la cantidad de dependencias: ¿Qué tamaño tiene la red? ¿Se puede acotar su flujo?

$$n = N + 2, \qquad m \leq N + R, \qquad F \leq P = \sum_{w_i > 0} w_i.$$

Complejidad

Si N es la cantidad de proyectos y R la cantidad de dependencias: ¿Qué tamaño tiene la red? ¿Se puede acotar su flujo?

$$n = N + 2, \qquad m \leq N + R, \qquad F \leq P = \sum_{w_i > 0} w_i.$$

Armar la red es fácil (lineal). Si usamos EK para resolver el modelo de flujo: ¿Cuál es la complejidad final?

$$O(\min\{mF, nm^2\}) = O((N + R) \min\{P, N(N + R)\})$$

Complejidad

Si N es la cantidad de proyectos y R la cantidad de dependencias: ¿Qué tamaño tiene la red? ¿Se puede acotar su flujo?

$$n = N + 2, \qquad m \leq N + R, \qquad F \leq P = \sum_{w_i > 0} w_i.$$

Armar la red es fácil (lineal). Si usamos EK para resolver el modelo de flujo: ¿Cuál es la complejidad final?

$$O(\min\{mF, nm^2\}) = O((N + R) \min\{P, N(N + R)\})$$

En general, si P es muy grande la segunda cota va a ser mejor. Como una expresión más linda, podemos acotar todo por

$$O(N(N + R)^2)$$

Conclusiones

- Introducimos el **problema de flujo**, que es particularmente útil para **modelar** muchísimos problemas (en particular, de asignación).
- Dimos dos algoritmos que lo resuelven, basados en la técnica de búsqueda local. Puntualmente, los algoritmos construyen flujos y los van mejorando a través de **caminos de aumento** de la **red residual**.
- En realidad, uno de los algoritmos (EK) es un versión más especificada de otro (FF). Por lo tanto, hereda la correctitud y la cota de complejidad.

Conclusiones

- Cosas que no vimos: hay mejores algoritmos de flujo basados en las ideas que mencionamos (buscar caminos de aumento). Por ejemplo, el de **Dinics**, que es $O(N^2M)$.
- Incluso hay una familia de algoritmos de flujo bastante distinta llamada **push-relabel** que alcanza complejidades mejores (con muy poco esfuerzo llegan a $O(N^3)$, y el mejor algoritmo que existe hoy en dia es $\sim O(MN \log n)$ y usa estas ideas).
- Están bien tratados en el Cormen :D